

Ticket Intake and Routing: Deliverables

Climb Senior Forward Deployed Engineer take-home · Austin Nafe · Repo:

github.com/anafe03/climb-intake

Functional requirements

✓ Process the 10 Climb-provided sample tickets

The tickets are in [data/sample_tickets.json](#). Load them with `POST /tickets/load-samples` or, in the app, Data then "The 10 Climb samples".

✓ Accept tickets via an API or batch upload and extract customer, category, urgency and an escalation flag

One ticket: `POST /tickets`. A batch: `POST /tickets/batch`. A file (.json, .jsonl, .csv or .txt): `POST /tickets/upload`. All in [app/main.py](#). The model reads each ticket in [app/llm_openai.py](#); the fields it must return are defined in [app/models.py](#).

✓ Route each ticket to a simulated downstream queue based on category and urgency

The routing table is [app/routing.yaml](#), applied by [app/routing.py](#). `GET /queues` lists every queue and how many tickets are in it.

✓ Flag escalations (security, legal/contract, executive mentions) with a low tolerance for misses

The model reads every ticket; then keyword rules in [app/rules.py](#) run as a safety net that can add an escalation but never remove one. **Every must-escalate ticket caught**, both with the model ([docs/EVAL-llm.md](#)) and with keyword rules alone ([docs/EVAL-rules.md](#)), on the 33-ticket test set [data/gold.jsonl](#). Automated test: [tests/test_gold_rules_mode.py](#).

✓ A simple view or API showing how a ticket was classified and why

The app at `http://localhost:8080`: click a ticket to see the reason behind each field ([app/static/index.html](#)). The same explanation as plain text: `GET /tickets/{id}/explain`.

Infrastructure requirements

✓ Containerize the service and provide a cloud deploy plan

[Dockerfile](#) and [docker-compose.yml](#). The deploy plan is in [README.md](#) under "Deploy plan (GCP Cloud Run)", with the full runbook in [docs/DEPLOY.md](#).

✓ **Log every routing decision, with the reasoning, for audit**

[app/audit.py](#) saves every decision to a database: the model's raw answer, which keyword rules fired and what they changed, and the time and cost of the call. Each decision is also written as a JSON line to the container logs. The full record: `GET /tickets/{id}`. Example records for all 10 samples, from a real run: [data/recorded/samples.json](#).

Deliverables

✓ **A containerized version that runs locally**

Run `docker compose up`, then open `http://localhost:8080`. Files: [Dockerfile](#), [docker-compose.yml](#).

✓ **Source code in a git repository**

Public on GitHub: github.com/anafe03/climb-intake

✓ **Deployment configuration / IaC used to stand it up in a cloud**

Terraform for Google Cloud Run: [infra/terraform/main.tf](#) sets up Artifact Registry, Secret Manager, a service account and Cloud Run. One-command deploy: [scripts/deploy.sh](#). It validates, but has not been applied to a live Google Cloud project.

✓ **A README with a plan for deploying to a cloud**

[README.md](#), "Deploy plan (GCP Cloud Run)"; details in [docs/DEPLOY.md](#).